Name: G. Balasubramanian

Reg No: 192521001

Department: B. Tech IT

Course Code & Name: CSA0304 – Data Structures

Faculty name: H.D. Manoj

Assignment - 01

1) A Data Structure is a method of organizing, storing, and managing data in a computer so that it can be accessed, modified, and processed efficiently.

Examples: Array, Linked List, Stack, Queue, Tree, and Graph.

Importance of Data Structures in Computing:

1. Efficient Data Management: Organizes large amounts of data systematically.
2. Faster Data Access: Helps search and retrieve data quickly.
3. Memory Utilization: Uses computer memory efficiently.
4. Improved Performance: Reduces the time required for operations such as searching, sorting, insertion, and deletion.
5. Problem Solving: Helps develop efficient algorithms for applications such as databases, operating systems, and network routing.

Conclusion: Data structures are essential in computing because they improve data organization, memory usage, and program efficiency.

2) Recursive Binary Search

1. Definition

Binary Search is a searching algorithm used to find an element in a sorted array by repeatedly dividing the search interval into two halves. The recursive method calls the same function until the element is found or the search interval becomes empty.

2. Algorithm

1. Set low = 0 and high = n - 1.
2. Calculate the middle index: mid = (low + high) / 2.
3. If the middle element equals the key, return its index.
4. If the key is smaller, search the left half recursively.
5. If the key is larger, search the right half recursively.
6. If low > high, return -1 (element not found).

3. C Program

```
#include <stdio.h>
int binarySearch(int a[], int low, int high, int key) {
    if (low > high)
        return -1;
    int mid = low + (high - low) / 2;
    if (a[mid] == key)
        return mid;
    else if (key < a[mid])
        return binarySearch(a, low, mid - 1, key);
    else
        return binarySearch(a, mid + 1, high, key);
}
int main() {
    int a[] = {10, 20, 30, 40, 50, 60, 70};
    int n = sizeof(a) / sizeof(a[0]);
    int key = 50;
    int result = binarySearch(a, 0, n - 1, key);
    if (result == -1)
        printf("Element not found");
    else
        printf("Element found at index %d", result);
```

```
    return 0;
}
```

4. Example and Execution

Given sorted array: {10, 20, 30, 40, 50, 60, 70}

Search key: 50

Step	Low	High	Mid	Middle element
1	0	6	3	40
2	4	6	5	60
3	4	4	4	50

Output:

Element found at index 4

5. Complexity Analysis

- Best-case time complexity: \(O(1)\)
- Average-case time complexity: \ (O (\log n) \)
- Worst-case time complexity: \(O(\log n)\)
- Auxiliary space complexity: \(O(\log n)\) due to recursive function calls.

6. Conclusion

Recursive binary search efficiently finds an element in a sorted array by dividing the search space into halves. It is faster than linear search for large sorted datasets.

3) ADT Usage in Database Indexing

1. Definition of ADT

An Abstract Data Type (ADT) is a logical model of a data structure that defines the data and the operations performed on it, without specifying how they are implemented.

2. ADT in Database Indexing

Database indexing uses ADTs to organize records and retrieve data quickly without scanning the entire database.

Examples of ADTs used in indexing:

1. Tree ADT: B-trees and B+ trees organize index keys for efficient searching, insertion, and deletion.
2. Hash ADT: Hash tables enable fast record retrieval using a key.
3. Set ADT: Maintains unique index keys when duplicate values are not allowed.
4. Map ADT: Associates index keys with record locations or pointers.
5. Priority Queue ADT: Supports priority-based retrieval in specialized database operations.

3. Example

Suppose a student database contains thousands of records. An index on Register number helps locate a student's record quickly instead of scanning every record.

4. Advantages

- Faster data retrieval.
- Efficient searching and updating.
- Reduced disk access in suitable indexing structures.
- Improved database performance.
-

Conclusion: ADTs provide a logical foundation for database indexing, helping databases organize records and access information efficiently.